

Attention Sinks

Why so many heads stare at the first token · video 2:17

This lesson answers one question: why do so many attention heads stare at the first token, and why does it matter? In GPT-2, 39% of all attention goes to the first token, whatever that token is. Softmax must add up to one, so a head with nothing useful to look at parks its attention on the one token every query can see: an **attention sink**, whose values are tiny. Drop it from a sliding-window cache and the model collapses (perplexity 48 → 2,133); keep the first four tokens and it recovers (43).

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. How much attention goes to the first token	▶ 0:08-0:53	2	☐
2. Why a sink, and why it is harmless	▶ 0:53-1:20	2	☐
3. Streaming, and four tokens that fix it	▶ 1:20-1:51	2	☐
4. The code	▶ 1:51-2:08	1	☐

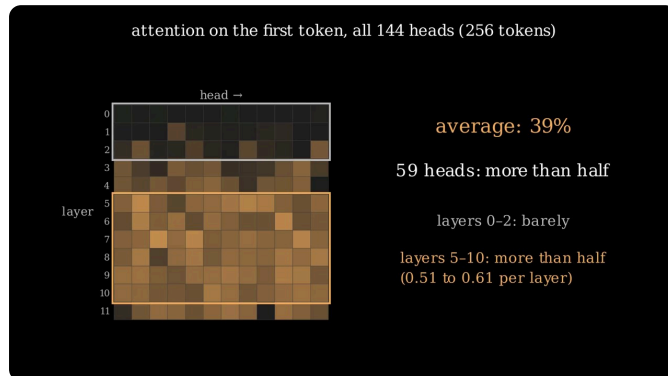
Before you start: Deep Dive episodes 8 (one head, entry by entry) and 11 (sliding windows). Code: `code/d12_attention_sinks` (GPT-2 small, about 500 MB).

CONCEPT 1 OF 4

How much attention goes to the first token

▶ Video 0:08-0:53

GPT-2 reads 256 tokens of Shakespeare. Averaged over all 144 heads (queries 16-255), **39%** of the attention goes to the very first token, and **59 heads** give it more than half. The first layers barely do it (0.01-0.16 per layer); layers 5 to 10 give it more than half (0.51 to 0.61).



▶ 0:38 Attention on the first token in all 144 heads: 39% on average, 59 heads above half, little in layers 0-2.

change the first token

line break	0.394
zebra	0.392
The	0.394
"," (a comma)	0.393
text cut mid-sentence	0.395

not the word: the position

▶ 0:52 Change the first token (line break, zebra, The, comma) or cut mid-sentence: always about 0.39.

Is the first token special? Replace it with a line break, *zebra*, *The* or a comma: 0.394, 0.392, 0.394, 0.393. Start the text mid-sentence: 0.395. It is not the word; it is the **position**.

KEY IDEA

Many heads send a large share of their attention to the first position, regardless of its content.

Check 1: How much attention goes to the first token

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

If 39% of attention goes to the first token on average, how much is left for the other 255 tokens together?

1.2 TRUE OR FALSE

"GPT-2 attends to the first token because it is usually an important word like "The"."

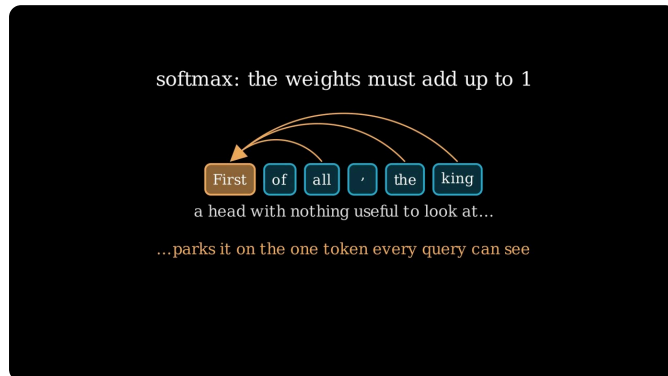
True ☐ False ☐

CONCEPT 2 OF 4

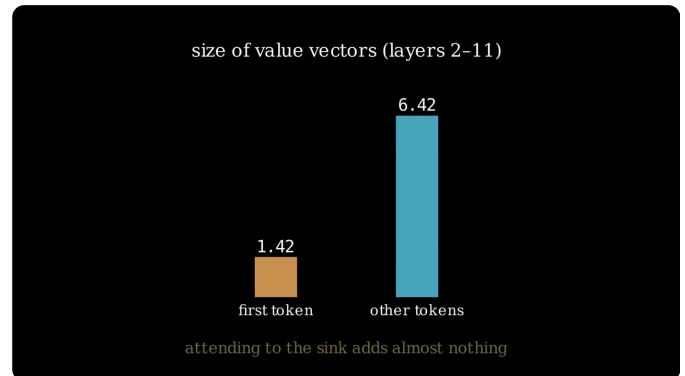
Why a sink, and why it is harmless

▶ Video 0:53-1:20

Softmax must add up to one. A head that has nothing useful to look at for a given token still has to put its attention somewhere. The first token is the one every query can see (because of the causal mask), so the model learns to park attention there: an **attention sink**.



▶ 1:06 Softmax must add up to 1: a head with nothing to look at parks its attention on the token every query can see.



▶ 1:19 The first token's value vectors are small (1.42 vs 6.42), so attending to it adds almost nothing.

Parking there is almost free: the first token's value vectors are small, an average size of **1.42** against **6.42** for the other tokens (layers 2-11). Attending to the sink adds almost nothing to the output: it works as a “do nothing” option.

KEY IDEA

An attention sink is a learned way to say “nothing here” under the constraint that the weights must sum to 1.

Check 2: Why a sink, and why it is harmless

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 SHORT ANSWER

Why is the first token, rather than, say, the tenth, the one that becomes the sink?

2.2 MULTIPLE CHOICE

What makes attending to the sink nearly a no-op?

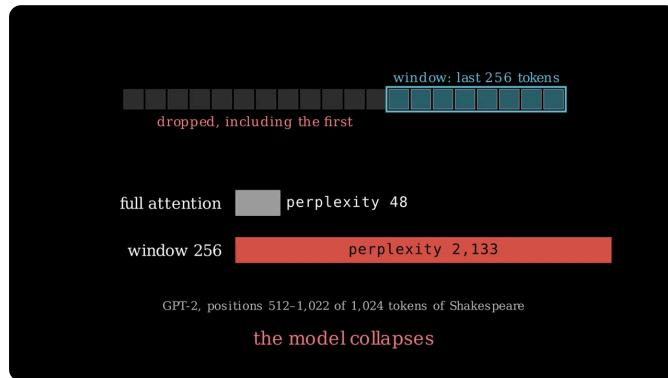
- A. Its scores are negative
- B. Its value vectors are small
- C. It is masked
- D. It has no position embedding

CONCEPT 3 OF 4

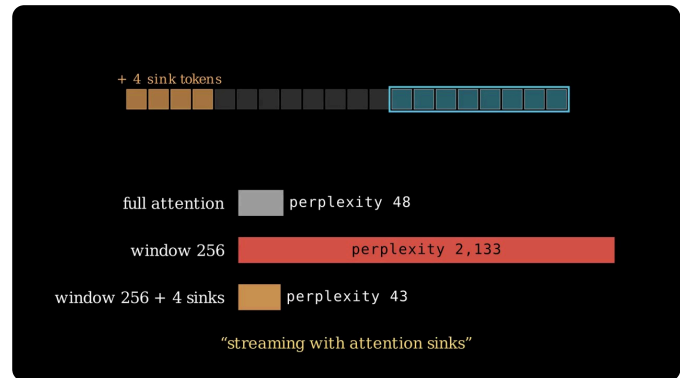
Streaming, and four tokens that fix it

▶ Video 1:20-1:51

To read an endless stream, a model can keep a sliding window: the last 256 tokens, dropping the oldest, including the first. On 1,024 tokens of Shakespeare (loss measured on positions 512-1,022), GPT-2 with full attention has a perplexity of **48**; with the window it jumps to **2,133**. The heads lose their sink and dump their attention on whatever tokens remain.



▶ 1:38 A sliding window of 256 that drops the first tokens: perplexity 48 → 2,133. The model collapses.



▶ 1:49 Keep the first four tokens plus the window: perplexity 43.

The fix: always keep the **first four tokens**, plus the window. Perplexity: **43**, even a little better than full attention on this text. The trick is known as streaming with attention sinks (StreamingLLM). Some newer designs give heads an explicit way to attend to nothing, such as a learned sink score for each head.

KEY IDEA

Keep the sink tokens in any cache that evicts old tokens; four tokens are enough to bring the model back.

Check 3: Streaming, and four tokens that fix it

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

With a window of 256 plus 4 sink tokens, at most how many keys does a query see?

3.2 HANDS-ON CODE

Try it yourself. In `attention_sinks.py`, keep only the first token (or the first two) instead of four. What perplexity do you get?

```
for n in (1, 2):
    mask = window | ((idx[None] < n) & (idx[None] <= idx[:, None]))
    lg, _, _ = forward(ids, mask)
    loss = torch.nn.functional.cross_entropy(lg[0, 512:-1], targets[512:])
    print(n, loss.exp())
```

CONCEPT 4 OF 4

The code

▶ Video 1:51-2:08

In code, sinks are one more term in the attention mask: a key is allowed if it is in the window, or if it is one of the first four tokens (and not in the future). The episode's code writes GPT-2's forward pass by hand so it can take any mask; it matches `transformers` to within 2×10^{-4} in the logits.

```
1 window = (j <= i) & (i - j < W)      # the last W tokens
2 sinks = (j < 4) & (j <= i)          # ... plus the first four, always
3 allowed = window | sinks
```

▶ 2:06 One more term in the mask: allowed if in the window, or one of the first four tokens.

KEY IDEA

Masks are the lever for all of part 3: causal, windowed, sparse, or windowed plus sinks.

Check 4: The code

Answer all 1 questions before looking at the answers. Pass mark: all correct.

4.1 PUT IN ORDER

Order these masks from fewest to most allowed pairs on 1,024 tokens: *full causal* · *window 256* + *4 sinks* · *window 256*.

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. How much attention goes to the first token · rewatch ▶ 0:08-0:53

1.1 61%. Each row of weights sums to 1 (100%).

1.2 False. With a line break, “zebra” or a comma as the first token, the share stays at about 0.39.

2. Why a sink, and why it is harmless · rewatch ▶ 0:53-1:20

2.1 Because of the causal mask, the first token is the only one visible to every query, including the earliest ones; a sink must be reachable from everywhere. A token at position 10 is invisible to queries 0-9.

2.2 B. The output is a weighted sum of values; a small value contributes little, whatever its weight.

3. Streaming, and four tokens that fix it · rewatch ▶ 1:20-1:51

3.1 260. The last 256 tokens plus tokens 0-3 (when they are not already in the window).

3.2 1 sink: perplexity 47; 2 sinks: 45 (4 sinks: 43). Even a single sink token prevents the collapse; GPT-2's sink is mostly the very first position.

4. The code · rewatch ▶ 1:51-2:08

4.1 window 256 → window 256 + 4 sinks → full causal. The sinks add a few pairs to the window; full causal allows every earlier key.